Basics

Program, types, I/O

Program

`#include <iostream>`	include header
`using namespace std;`	skip std::
`int main() { return 0; }`	entry point
`std::cout << "Hi" << std::endl;`	print
`std::cin >> x;`	read input
`std::getline(std::cin, s);`	read full line
`// one line  /* many */`	comments

Types

`int  long  short`	whole numbers
`float  double`	decimals
`char  bool`	character / true-false
`unsigned int`	no negatives
`auto x = 5;`	inferred type
`const int N = 10;`	constant
`constexpr int N = 10;`	compile-time value
`sizeof(int)`	size in bytes

Operators

`+  -  *  /  %`	math
`==  !=  <  >`	compare
`&&  ||  !`	logic
`&  |  ^  <<  >>`	bitwise
`++  --  +=`	shortcuts

I/O & Files

Streams and formatting

Console I/O

`std::cout << x << '\n';`	print a value
`std::cerr << "error";`	print to error stream
`std::cin >> a >> b;`	read two values
`std::getline(std::cin, s);`	read full line
`std::cin.ignore();`	skip leftover newline
`std::ios::sync_with_stdio(false);`	faster I/O

Formatting (<iomanip>)

`std::setw(8)`	field width
`std::setprecision(2)`	digits
`std::fixed`	fixed decimals
`std::setfill('0')`	pad character
`std::boolalpha`	print true / false
`std::hex  std::dec`	number base

Files (<fstream>)

`std::ifstream in("a.txt");`	open for reading
`std::ofstream out("a.txt");`	open for writing
`std::ofstream out("a", std::ios::app);`	append mode
`while (std::getline(in, line)) { }`	read line by line
`if (!in) { }`	open failed?
`in.close();`	close file

Numbers & Math

cmath, limits, random

<cmath>

`std::sqrt(x)  std::pow(a, b)`	root / power
`std::abs(x)  std::fabs(x)`	absolute value
`std::floor(x)  std::ceil(x)`	round down / up
`std::round(x)  std::trunc(x)`	nearest / cut
`std::fmod(a, b)`	float remainder
`std::log(x)  std::exp(x)`	log / e to x
`std::sin(x)  std::cos(x)`	trigonometry

Limits & Types

`INT_MAX  INT_MIN`	int limits
`std::numeric_limits<int>::max()`	type limit
`int64_t  uint8_t`	fixed-size ints
`size_t`	size / index type
`static_cast<double>(x)`	safe conversion

Random & Numeric

`std::mt19937 gen(seed);`	random engine
`std::uniform_int_distribution<int> d(1, 6);`	range
`d(gen)`	draw a number
`std::gcd(a, b)  std::lcm(a, b)`	gcd / lcm
`std::iota(v.begin(), v.end(), 0);`	fill 0,1,2...

Strings & Vectors

STL containers

string

`#include <string>`	string header
`s.length()  s.size()`	length
`s.substr(1, 3)`	part of string
`s.find("a")`	index or npos
`s.append("x")  s += "x"`	add text
`s.replace(0, 1, "b")`	replace part
`std::to_string(5)`	number to text
`std::stoi("5")`	text to int

vector

`std::vector<int> v = {1, 2};`	dynamic array
`v.push_back(3)`	add to end
`v.pop_back()`	remove last
`v.size()  v.empty()`	size / empty?
`v[0]  v.at(0)`	access (at checks)
`v.insert(v.begin(), 9)`	insert
`v.erase(v.begin())`	remove
`v.clear()`	empty it

Others

`std::array<int, 3> a;`	fixed array
`std::pair<int,int> p;`	two values
`std::tuple<int,string> t;`	many values

Strings II

Chars and views

Characters (<cctype>)

`std::isdigit(c)  std::isalpha(c)`	digit / letter?
`std::isspace(c)  std::isalnum(c)`	space / alnum?
`std::toupper(c)  std::tolower(c)`	change case
`c - '0'`	digit char to int
`char(c + 1)`	next character

Searching & Editing

`s.find("a", pos)`	search from pos
`s.rfind("a")`	last occurrence
`s.find_first_of("abc")`	first of any char
`s.starts_with("a")`	prefix? (C++20)
`s.ends_with("z")`	suffix? (C++20)
`s.insert(0, "x");`	insert text
`s.erase(0, 2);`	remove part
`s.compare(t)`	0 if equal

Views & Streams

`std::string_view sv = s;`	no-copy view
`sv.substr(1, 3)`	view of a part
`std::stringstream ss(s);`	text as a stream
`ss >> a >> b;`	split by spaces
`std::getline(ss, t, ',');`	split by comma
`std::to_string(3.14)`	number to text
`std::stod("3.14")`	text to double

Maps & Algorithms

Containers and tools

map & set

`std::map<string,int> m;`	sorted key-value
`std::unordered_map<K, V>`	hash map
`m["a"] = 1;`	set value
`m.count("a")  m.find("a")`	key exists?
`std::set<int> s;  s.insert(1)`	unique sorted
`std::stack  std::queue`	LIFO / FIFO
`std::priority_queue<int>`	max heap

Iterate

`for (int x : v) { }`	range for
`for (auto& x : v) { }`	by reference
`for (auto it = v.begin(); it != v.end(); ++it)`	iterator

<algorithm>

`std::sort(v.begin(), v.end());`	sort
`std::reverse(v.begin(), v.end());`	reverse
`std::find(v.begin(), v.end(), x)`	find item
`std::count(v.begin(), v.end(), x)`	count item
`std::max(a, b)  std::min(a, b)`	larger / smaller
`std::accumulate(b, e, 0)`	sum

Algorithms II

Iterators and lambdas

Search & Check

`std::binary_search(b, e, x)`	in sorted range?
`std::lower_bound(b, e, x)`	first >= x
`std::any_of(b, e, f)`	any match?
`std::all_of(b, e, f)`	all match?
`std::none_of(b, e, f)`	no match?

Modify

`std::transform(b, e, out, f)`	map each item
`std::remove_if(b, e, f)`	move bad items back
`v.erase(std::remove(b, e, x), e);`	erase-remove idiom
`std::unique(b, e)`	drop neighbor duplicates
`std::fill(b, e, 0)`	set all to 0
`std::rotate(b, m, e)`	rotate range

Sort & Compare

`std::sort(b, e, std::greater<>());`	sort descending
`std::sort(b, e, cmp);`	custom order
`std::max_element(b, e)`	iterator to max
`std::min_element(b, e)`	iterator to min
`std::next_permutation(b, e)`	next ordering
`std::equal(b, e, b2)`	ranges equal?

Control & Functions

Flow and functions

Control Flow

`if (x) { } else if (y) { } else { }`	branches
`x ? a : b`	ternary
`switch (x) { case 1: break; }`	many cases
`for (int i = 0; i < 5; i++)`	classic loop
`while (cond) { }`	loop while true
`do { } while (cond);`	run at least once
`break;  continue;`	exit / skip

Functions

`int add(int a, int b) { }`	define function
`vold f(int& a)`	by reference
`void f(const string& s)`	read-only reference
`int f(int a = 5)`	default value
`auto f = [](int a) { return a * 2; };`	lambda
`[&](int x) { }`	capture by reference
`template<typename T> T f(T a)`	generic function

Errors

`try { } catch (const exception& e) { }`	handle error
`throw runtime_error("msg");`	throw error

Templates & Lambdas

Write generic code

Templates

`template<typename T>`	generic header
`T maxOf(T a, T b) { }`	generic function
`template<class T> class Box { };`	generic class
`template<int N>`	value parameter
`Box<int> b;`	use a template
`requires std::integral<T>`	constraint (C++20)

Lambdas

`[](int a) { return a * 2; }`	simple lambda
`[=](int a) { }`	capture by copy
`[&](int a) { }`	capture by reference
`[x, &y](int a) { }`	pick captures
`[](auto a) { return a; }`	generic lambda
`[]() mutable { }`	change copied values

std::function & auto

`std::function<int(int)> f;`	store any callable
`auto f = [](int a) { };`	keep a lambda
`using Fn = void(*)(int);`	function pointer type
`std::bind(f, 1, _1)`	fix an argument

Classes

OOP in C++

Class

`class A { public: int x; };`	class
`A(int x) : x(x) { }`	constructor list
`~A() { }`	destructor
`private: protected: public:`	access
`void f() const;`	doesn't change object
`static int count;`	shared member
`this->x`	current object

Inheritance

`class B : public A { };`	inherit
`virtual void f();`	can override
`void f() override;`	safe override
`virtual void f() = 0;`	abstract
`virtual ~A() = default;`	safe delete

Operators & Friends

`A operator+(const A& o)`	overload +
`friend ostream& operator<<(...)`	custom print
`struct P { int x; };`	public by default
`enum class C { Red, Blue };`	scoped enum

Modern C++

C++11 to C++20

Language Features

`auto x = 5;`	type deduction
`for (auto& [k, v] : m) { }`	structured bindings
`if (auto it = m.find(k); it != m.end())`	if with init
`constexpr int sq(int x) { }`	compile-time function
`nullptr`	safe null pointer
`enum class Color { Red };`	scoped enum

Library Features

`std::optional<int> o;`	maybe a value
`std::variant<int, string> v;`	one of several types
`std::any a = 5;`	hold any type
`std::span<int> s(v);`	view of an array
`std::format("{} {}", a, b)`	type-safe format (C++20)
`std::filesystem::exists(p)`	file exists?

Handy Syntax

`using Vec = std::vector<int>;`	type alias
`auto [a, b] = pair;`	unpack a pair
`[[nodiscard]] int f();`	warn if ignored
`x <=> y`	three-way compare
`for (int x : v | std::views::reverse)`	ranges (C++20)

Pointers & Memory

Handle with care

Pointers

`int* p = &x;`	pointer to x
`*p`	value at pointer
`p->member`	member via pointer
`int& r = x;`	reference
`nullptr`	null pointer
`int* a = new int[5];`	heap array
`delete p;  delete[] a;`	free memory

Smart Pointers

`#include <memory>`	smart pointers
`std::unique_ptr<A> p = std::make_unique<A>();`	single owner
`std::shared_ptr<A> p = std::make_shared<A>();`	shared owner
`std::weak_ptr<A>`	non-owning
`std::move(x)`	transfer ownership

Rules

`RAII`	resources tied to scope
`Rule of 0 / 3 / 5`	copy & move rules

Move & Exceptions

Cheap copies, safe code

Move Semantics

`A(A&& o) noexcept`	move constructor
`A& operator=(A&& o) noexcept`	move assignment
`std::move(x)`	allow moving from x
`std::forward<T>(x)`	keep value category
`A(const A&) = default;`	default copy
`A(const A&) = delete;`	forbid copying

Exceptions

`try { } catch (...) { }`	catch anything
`catch (const std::exception& e)`	catch by reference
`e.what()`	error message
`throw;`	rethrow current error
`std::runtime_error  std::logic_error`	common errors
`std::out_of_range  std::invalid_argument`	typed errors
`noexcept`	promise: no throw

RAII Helpers

`std::lock_guard<std::mutex> g(m);`	auto unlock
`std::unique_ptr<FILE, decltype(&fclose)>`	auto close file
`struct Guard { ~Guard() { } };`	cleanup in destructor
`assert(x > 0);`	debug check
`static_assert(sizeof(int) == 4);`	compile-time check

Concurrency & Time

Threads and chrono

Threads

`std::thread t(f, arg);`	start a thread
`t.join();`	wait for it
`t.detach();`	let it run alone
`std::this_thread::sleep_for(100ms);`	pause
`std::thread::hardware_concurrency()`	number of cores

Sync

`std::mutex m;`	a lock
`std::lock_guard<std::mutex> g(m);`	lock in scope
`std::atomic<int> n{0};`	lock-free counter
`std::condition_variable cv;`	wait for an event
`std::async(f, x)`	run, get a future
`fut.get()`	wait for result

<chrono>

`auto t0 = std::chrono::steady_clock::now();`	start timer
`std::chrono::duration<double>(t1 - t0)`	elapsed seconds
`std::chrono::milliseconds(50)`	a duration
`using namespace std::chrono_literals;`	50ms, 2s

Tips & Compile

Build and habits

g++ / clang++

`g++ main.cpp -o app`	compile
`g++ -std=c++20 -Wall -O2 main.cpp`	standard + warnings
`g++ -g main.cpp`	debug symbols
`./app`	run program

CMake

`cmake -B build`	configure
`cmake --build build`	build

Good Habits

`Prefer smart pointers`	no manual delete
`Use const & references`	avoid copies
`Prefer vector over arrays`	safer, flexible

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily